
Big Data

Management and Analytics

TU Clausthal, Institut für Informatik

Summer Semester 2026

Due Date: 10 June 2026, 16:00

Instructions — Please Read Carefully

In this assignment, we study edge betweenness and community detection. In Task 1, you manually calculate edge betweenness on a given graph without using programming and explain how it can be used to identify communities. In the second task, you need to familiarize yourself with GDS libraries on Neo4j. Before starting, you should become familiar with the syntax of centrality algorithms. You should clearly show all steps, explain your reasoning, and state any assumptions if information is missing. ***You must install the GDS plugin into your Neo4j database instance***

Cypher Reference: [click here](#)

GDS Reference: [click here](#)

Assignment Rules

- The assignment has a total of **10 marks**
- Work in your assigned teams of **up to 2 members**
- Submit your solutions **via Moodle**
- Solutions submitted after the deadline will automatically receive **0 marks**
- Copying solutions from or sharing with other students will lead to **exclusion from the course**
- The use of language software is **not permitted**; each group member must understand every part of the solution
- Document your solutions with **screenshots** and attach a **TXT file** with your code/queries
- If anything is unclear, state your assumptions explicitly to make your solution understandable

Task 1 **5 Marks**

This task focuses on **edge betweenness** and **community detection**. Use the graph provided in **Fig. 1: G** to answer the following questions.

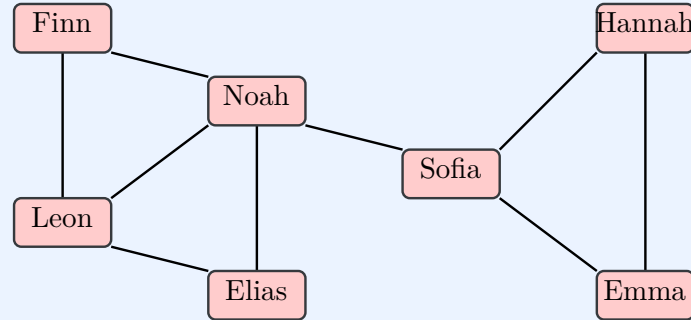


Fig. 1: G

- (a) For each node n_i in the graph G , perform a **Breadth-First Search (BFS)** using n_i as the root node (starting point). The result is a collection of DAGs, each using a different node as its root. *The subsequent graphs must be well formatted (drawn) such that each level can be recognized just by looking at them.*
- (b) For all graphs created in Task 1(a), label each node by the number of shortest paths that reach it from the root. The root node is labeled with 1.
In one sentence, explain what these numbers indicate.
- (c) Identify each leaf node and assign a credit value of 1 to it.
- (d) For all non-leaf nodes, compute the node credit as

$$1 + (\text{sum of the credits received from its children}).$$

When a node Z has multiple parents, split the credit of Z among the incoming edges according to how many shortest paths reach Z through each parent.

- (e) Make a copy G_{copy} of the graph G . For each edge in G , sum the labels of that edge across all graphs in Task 1(d). Write down half of this sum as the new label on G_{copy} . The result is the edge betweenness of each edge in G .
- (f) In your own words, explain edge betweenness centrality. What role does edge betweenness play in detecting communities in graphs? From the resultant graph in Task 1(e), explain how you would create communities and draw the resultant communities created after the first iteration.

Task 2 5 Marks

This task focuses on **centrality measures** in the **E-Commerce** database using Neo4j and the Graph Data Science (GDS) library. Write the required Cypher/GDS queries and briefly explain your reasoning and results.

- (a) How do you interpret the concept of **betweenness centrality** in the context of the e-commerce database? What would it mean for a **Product** or a **User** to have high betweenness centrality in this graph, and how does this differ from its interpretation in a classic social network?
- (b) Given the following GDS query:

```
CALL gds.betweenness.stream(  
  graphName: String,  
  configuration: Map  
)  
YIELD nodeId: Integer, score: Float
```

Explain this query in detail. What are the parameters `graphName` and `configuration`? What does the query do, and what do the output fields `nodeId` and `score` represent?

- (c) Create a graph projection that includes a subset of node labels and relationship types from the database of your choice. **Justify why you selected those specific nodes and relationships.** Then execute the betweenness centrality algorithm on your projected graph using at least one non-default configuration parameter, such as `samplingSize` or `concurrency`, and **explain why you chose that value.** Finally, interpret the results.
- (d) Calculate the **closeness centrality** for every **User** and every **Product** in the database. Your result should be a table with two columns: the entity name and its closeness centrality score.
- Then answer the following:
- What is the difference between betweenness and closeness centrality?
 - What would high closeness centrality mean for a **User** versus a **Product** in this e-commerce context?

Deliverables: Submit a `.txt` file containing all Cypher/GDS queries and screenshots showing the execution results in Neo4j.